

Group project - Wine - Jared Martin - R Code

Jared Martin

2024-11-24

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(class)
library(tree)
library(randomForest)
```

```
## randomForest 4.7-1.1
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
## The following object is masked from 'package:ggplot2':
##
##   margin
```

```
library(e1071)
library(nnet)
library(plotly)
```

```
##
## Attaching package: 'plotly'
##
## The following object is masked from 'package:ggplot2':
##
##   last_plot
```

```
##
## The following object is masked from 'package:stats':
##
##     filter
##
## The following object is masked from 'package:graphics':
##
##     layout
```

```
library(knitr)
```

Group Members: 1 of 1 Jared Martin

Cleaning and Preparing the data

The following code pulls in the two wine data sets, creates a new variable called wine type (red or white), makes quality and type factor variables, joins the two data sets together and saves the data as a csv.

```
rbind(read.csv("./Data/winequality-red.csv", sep = ";") %>%
  mutate(type = "red"),
  read.csv("./Data/winequality-white.csv", sep = ";") %>%
  mutate(type = "white")
) %>%
write.csv( "./Data/wine_data.csv")

wine_data <- read_csv("./Data/wine_data.csv")
```

```
## New names:
## Rows: 6497 Columns: 14
## -- Column specification
## ----- Delimiter: "," chr
## (1): type dbl (13): ...1, fixed.acidity, volatile.acidity, citric.acid,
## residual.sugar...
## i Use 'spec()' to retrieve the full column specification for this data. i
## Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## * ' -> '...1'
```

```
wine_data$quality <- factor(wine_data$quality)

wine_data[,2:14] -> wine_data

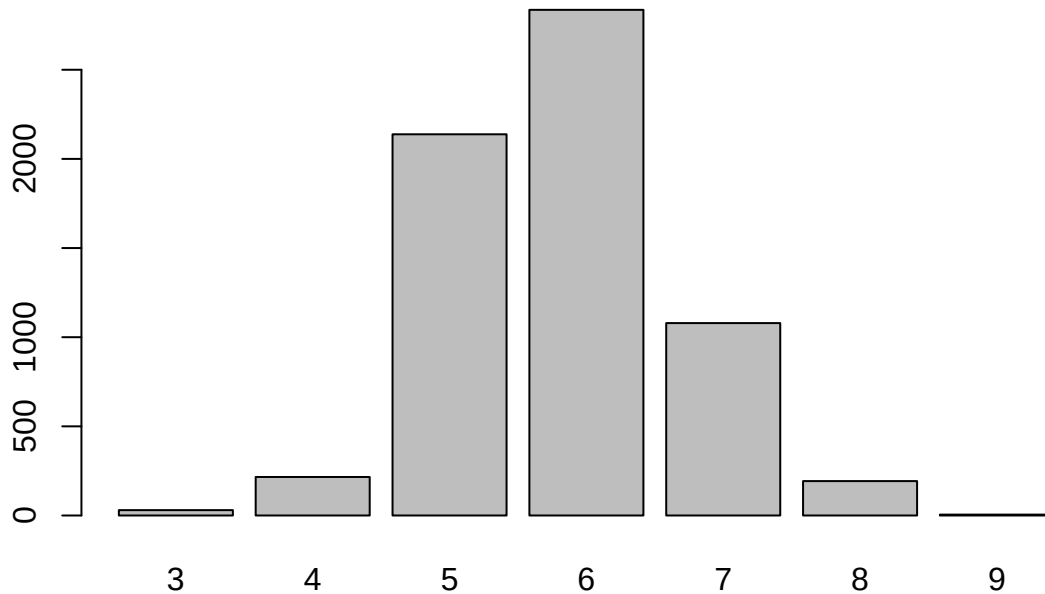
wine_data %>%
  dplyr::select(quality, everything()) -> wine_data
```

Research Question #1: What are the characteristics of good Portuguese wine?

Method #6: Clustering - K Means

1. Summary Statistics

```
plot(wine_data$quality)
```



```
set.seed(1)

wine_data_numeric <- wine_data %>%
  select(-type)

wine_data_numeric$quality <- as.numeric(wine_data_numeric$quality)

custom_summary <- wine_data_numeric %>%
  summarise_all(list(
    Mean = ~ mean(., na.rm = TRUE),
    Median = ~ median(., na.rm = TRUE),
    SD = ~ sd(., na.rm = TRUE),
    Min = ~ min(., na.rm = TRUE),
    Max = ~ max(., na.rm = TRUE),
    shapirowilks = ~ shapiro.test(sample(., size = 4999))$p.value
  )) %>%
  pivot_longer(cols = everything(),
    names_to = c("Variable", "Statistic"),
    names_sep = "_") %>%
  pivot_wider(names_from = Statistic, values_from = value) %>%
  write_csv( "./Data/summary.csv")
```

Quality categorical variable follows a fairly normal distribution; therefore, Converting the variable back into

a numeric value to cluster. The clusters with the highest average quality score should have the highest portion of high quality wine. We see the characteristics the wine critics are looking for in a good wine.

2. K-Means Clustering

```
kmeans_result <- kmeans(wine_data_numeric, 6)

kmeans_result

## K-means clustering with 6 clusters of sizes 1084, 1608, 1081, 398, 937, 1389
##
## Cluster means:
##   quality fixed.acidity volatile.acidity citric.acid residual.sugar  chlorides
## 1 3.720480      8.397878      0.5077030    0.2696494      2.474308 0.08249446
## 2 4.019279      6.853545      0.2920056    0.3182525      4.584546 0.04508085
## 3 3.653099      6.949399      0.2857771    0.3513691      8.719473 0.05134413
## 4 3.522613      7.009799      0.3077387    0.3565075     10.033291 0.05224121
## 5 3.744931      7.354749      0.3890768    0.2951334      2.969424 0.06273212
## 6 3.925126      6.882973      0.2814579    0.3368251      6.558135 0.04828150
##   free.sulfur.dioxide total.sulfur.dioxide  density      pH sulphates
## 1      10.73662      27.36900 0.9964440 3.304400 0.6399354
## 2      27.32245     109.59049 0.9930004 3.198483 0.4955037
## 3      46.52498     177.45745 0.9958282 3.185504 0.5058094
## 4      55.12814     221.84296 0.9967830 3.176910 0.5182915
## 5      21.05977      73.16222 0.9939189 3.240107 0.5586126
## 6      36.56048     142.13463 0.9943428 3.197660 0.4929518
##   alcohol
## 1 10.587654
## 2 10.936806
## 3  9.867730
## 4  9.556533
## 5 10.837602
## 6 10.422232
##
## Clustering vector:
##   [1] 1 5 5 1 1 5 1 1 2 5 2 5 1 6 6 2 5 1 5 5 1 5 1 1 1 1 1 1 5 1 2 5 1 1 1
##  [38] 1 1 5 5 1 1 1 1 5 2 1 1 2 1 1 1 2 5 1 1 2 5 1 5 2 1 5 1 1 1 1 5 1 1 2 2 1
##  [75] 5 1 1 1 5 2 1 5 5 5 5 1 6 1 2 1 6 6 6 1 5 2 1 1 1 1 1 1 1 5 1 5 5 5 6 1
## [112] 2 2 1 1 5 1 1 5 5 5 5 1 1 2 5 1 1 1 1 6 2 2 1 1 5 5 1 2 2 5 5 5 1 5 6 2 5
## [149] 1 1 1 5 2 2 6 6 6 6 1 5 1 1 1 2 2 2 2 1 1 5 1 1 1 1 1 1 1 1 1 1 1 2 1 5 5
## [186] 5 5 1 6 6 2 1 2 1 1 2 5 1 5 1 1 6 1 1 1 1 1 2 2 1 1 5 1 5 1 2 1 1 1 6 5 5
## [223] 1 1 5 5 5 1 5 1 2 1 5 1 1 1 1 1 1 1 5 1 2 1 1 1 5 5 1 1 1 1 1 5 1 2 1 5 1
## [260] 1 1 5 1 5 1 1 5 1 1 1 5 1 5 1 2 5 1 1 1 1 1 1 5 1 5 5 5 5 1 2 1 1 1 1 5 1
## [297] 5 1 1 1 1 1 1 1 5 1 5 1 1 1 1 2 2 6 5 5 2 5 5 5 5 5 1 5 1 1 1 1 1 1 1 2
## [334] 1 1 1 1 5 2 1 1 1 1 1 1 5 1 1 1 1 1 1 1 5 3 1 5 1 1 1 5 5 1 1 1 1 1 1 5 1
## [371] 5 1 1 5 1 1 1 1 1 5 1 1 1 1 5 1 1 1 5 1 5 1 1 2 1 1 6 1 1 1 6 1 5 1 1 1 1
## [408] 1 1 5 5 5 5 1 2 6 1 2 1 5 5 5 5 1 5 5 1 1 1 1 1 5 1 1 5 1 5 1 5 1 1 1 1 1
## [445] 1 1 5 1 1 1 1 1 1 1 5 1 1 5 1 1 1 1 1 6 1 1 5 1 1 5 1 5 5 1 1 1 1 1 1 1
## [482] 1 1 1 1 1 1 1 5 1 5 1 1 5 2 1 1 2 1 5 1 5 5 1 1 1 1 1 5 1 1 5 1 1 1 6 1 1
## [519] 1 2 1 1 6 6 2 5 2 5 5 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 5 1 2 1 1 5 1 1 1 2 1
## [556] 1 1 1 1 1 1 2 2 5 1 1 1 1 1 1 1 1 1 1 1 5 1 1 2 2 1 1 1 1 1 5 1 1 2 5 1 5 6
## [593] 5 1 1 2 1 1 1 1 1 1 1 1 5 1 1 5 5 1 5 1 1 1 2 5 5 1 1 1 2 2 1 1 5 5 1 1 1
```

```

## [630] 5 1 1 1 5 2 1 6 6 1 5 1 5 1 5 1 1 1 1 1 6 1 6 5 1 1 5 1 1 1 1 1 1 1 1 5 1
## [667] 1 1 1 1 1 1 6 1 1 1 1 5 1 1 1 5 1 6 1 1 1 1 1 1 5 5 2 2 1 1 1 5 1 5 1 1
## [704] 5 1 5 1 1 1 1 5 2 1 1 5 1 1 1 1 1 1 2 1 6 1 1 1 1 1 5 1 1 1 5 1 1 1 5 2 1
## [741] 1 6 1 2 2 1 5 5 1 1 1 1 5 1 1 1 1 1 1 5 2 1 1 1 1 1 5 2 2 1 2 6 6 1 1 1 1
## [778] 1 1 2 1 1 2 1 5 5 5 5 5 2 5 6 5 1 1 5 5 1 1 1 2 1 5 1 1 1 1 1 1 1 1 1 1 1
## [815] 1 1 1 1 1 5 1 1 1 1 1 1 1 1 1 1 1 1 5 5 1 5 2 2 1 1 1 1 5 5 1 1 1 1 1 1 1
## [852] 1 2 5 5 1 5 1 1 1 5 5 1 5 5 5 1 1 1 1 1 5 5 1 1 1 1 1 5 5 1 1 1 5 5 1 1 5
## [889] 5 2 5 5 1 5 5 1 1 1 1 1 1 1 1 1 1 2 5 1 5 1 1 1 1 1 1 1 5 1 5 5 1 5 5 1 1
## [926] 5 5 5 1 1 1 1 5 1 1 5 5 1 1 1 1 1 5 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [963] 1 1 1 1 1 2 1 1 1 1 1 1 1 5 5 2 1 1 1 1 5 1 1 1 1 5 1 1 1 5 1 5 5 1 1 1 1
## [1000] 1 1 5 1 1 1 1 1 1 1 5 1 1 1 1 1 1 1 2 2 1 1 1 1 1 1 1 1 1 5 5 1 1 1 1 1 1
## [1037] 1 5 1 1 1 1 1 1 5 1 1 5 1 1 5 1 1 1 5 5 1 5 5 1 1 1 1 1 1 1 1 1 1 1 5 1 2 5
## [1074] 1 2 5 1 1 1 4 1 4 5 5 5 5 1 1 1 1 5 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 5
## [1111] 1 5 1 1 5 1 1 1 1 1 1 1 1 1 1 1 1 5 5 5 1 6 1 1 1 5 1 1 2 5 5 5 1 1 5 5 1
## [1148] 1 1 1 1 5 1 1 5 1 2 2 5 1 1 1 1 1 1 1 5 5 1 1 1 1 1 5 5 5 1 1 5 1 1 1 5 5
## [1185] 2 1 1 1 2 1 1 1 1 1 5 5 2 1 5 2 1 1 1 5 1 1 1 5 1 1 1 5 1 1 1 1 5 5 1 5 1
## [1222] 1 5 1 1 5 1 1 5 5 5 2 5 1 1 2 1 1 1 1 5 5 1 5 3 1 1 1 1 1 1 5 1 1 1 1 5 5
## [1259] 1 1 5 1 5 1 5 1 1 1 5 5 5 5 1 1 1 5 1 1 5 1 5 5 1 5 1 1 1 1 2 2 1 5 1 1 5
## [1296] 5 5 1 1 1 1 5 1 5 5 5 5 1 5 5 5 1 1 5 5 5 1 1 5 5 5 1 1 5 1 1 1 1 1 5 5 5
## [1333] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 5 1 5 5 1 1 1 5 2 1 1 5 1 5 1 1 1 2 5
## [1370] 1 5 1 5 5 1 2 1 5 1 1 1 1 2 2 5 5 1 1 1 2 1 1 1 5 5 1 1 2 1 1 6 6 1 1 1 1
## [1407] 1 5 5 5 1 1 1 5 1 1 1 1 1 6 1 5 5 1 1 1 5 1 5 5 1 5 1 1 2 2 5 1 1 2 1 2 1
## [1444] 1 2 2 1 1 5 1 1 1 5 5 1 1 5 5 1 1 5 1 1 1 5 5 5 1 5 1 1 1 1 1 5 5 5 5 1 1
## [1481] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 6 1 1 6 1 1 1 1 5 5 1 1 1 1 1 1 1 5 1 1 5 5 5
## [1518] 1 1 1 1 1 5 5 1 1 1 1 5 5 1 1 1 5 1 1 1 1 1 5 1 1 1 1 1 1 1 1 1 1 5 1 1 5 1
## [1555] 1 1 1 1 2 6 6 6 1 1 1 1 5 1 1 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 1 2 1 1 5 1 2 5 1
## [1592] 1 1 1 1 5 1 1 1 3 6 2 3 3 2 6 3 6 6 5 2 5 6 3 2 2 5 3 6 5 2 2 3 6 4 6 6 6
## [1629] 2 6 1 2 2 3 2 6 6 6 6 3 3 6 6 6 3 4 4 6 6 3 2 5 2 6 3 4 6 5 2 2 6 6 5 2 2
## [1666] 2 3 2 2 4 4 3 5 2 2 5 5 2 2 2 3 3 4 6 6 3 4 6 6 3 4 6 6 6 4 3 5 5 3 3 3 3
## [1703] 3 3 2 3 3 3 3 3 4 3 6 6 2 6 5 3 3 5 6 6 6 6 6 2 4 3 3 5 4 4 3 4 3 2 6 2 5
## [1740] 5 6 2 2 5 2 3 2 5 2 6 6 2 2 5 3 3 2 2 2 6 5 3 4 3 3 2 6 2 6 2 2 6 3 6 5
## [1777] 3 2 3 3 3 3 4 4 4 3 2 2 4 4 3 2 2 3 3 3 4 4 4 3 4 4 6 2 6 6 2 5 3 5 2 6 2
## [1814] 2 6 3 6 3 3 3 3 2 6 6 6 3 4 3 6 6 3 4 3 3 3 3 4 2 6 3 5 2 4 6 4 6 5 5 2 3
## [1851] 3 6 2 6 6 2 2 2 5 2 6 2 4 3 6 3 6 6 6 3 6 6 2 4 3 3 5 5 2 2 2 4 3 3 3 3 3
## [1888] 3 3 3 6 3 6 3 3 3 3 6 2 5 5 5 3 3 3 3 3 6 6 2 3 6 3 3 6 6 6 6 5 5 6 6 6 3
## [1925] 4 4 6 3 5 2 6 5 6 5 5 2 3 6 6 6 2 6 6 6 6 5 3 2 6 6 6 6 6 4 3 3 3 6 3 3 5
## [1962] 6 3 2 2 6 6 3 5 6 3 3 6 5 5 2 2 6 2 2 3 6 6 2 5 2 4 6 3 3 5 2 5 3 3 2 2 3
## [1999] 5 2 3 2 4 3 3 2 2 2 2 6 6 5 5 6 6 2 4 2 6 6 4 4 4 3 4 4 3 5 3 3 5 4 6 2 2
## [2036] 4 3 4 2 2 6 6 4 6 5 6 2 2 6 3 6 6 2 6 2 5 2 4 4 2 3 3 5 3 6 6 5 3 3 6 3 5
## [2073] 6 2 4 2 2 6 3 3 2 6 3 4 2 2 5 3 3 5 5 3 6 2 3 3 6 3 3 4 3 4 4 6 6 6 3 6 3
## [2110] 6 6 6 5 2 3 6 6 5 1 6 6 2 2 5 6 1 2 2 2 2 6 3 3 3 3 3 3 5 4 3 3 6 6 3 6 3
## [2147] 5 6 3 3 2 2 3 2 2 6 6 3 6 6 6 3 6 3 2 1 6 6 3 3 2 3 6 6 3 3 6 2 6 3 2 3 5
## [2184] 6 5 6 6 2 6 6 2 6 2 2 6 6 2 2 5 6 2 2 2 6 6 6 6 3 2 2 2 6 6 6 6 2 3 3 6 4
## [2221] 3 6 5 6 6 3 4 4 2 6 6 6 3 6 6 6 6 3 3 6 3 3 3 3 3 6 6 3 3 3 3 3 6 6 2 2 6
## [2258] 3 4 5 6 3 5 3 6 3 3 3 4 3 2 6 2 4 3 4 2 2 2 3 3 3 3 3 4 2 3 4 2 6 3 3 3 4
## [2295] 3 2 4 4 4 3 6 5 2 2 1 4 3 6 2 3 6 6 4 3 6 4 6 6 6 3 3 2 5 2 6 2 2 6 3 6 4
## [2332] 1 3 3 2 3 3 6 2 1 5 6 6 6 2 4 3 3 3 3 3 3 3 4 6 2 3 3 6 6 3 3 4 6 3 6 4 1 2
## [2369] 2 2 6 6 3 2 2 2 3 3 6 5 4 3 6 3 2 2 2 2 2 2 6 2 3 6 4 6 2 5 3 4 4 3 2 3
## [2406] 4 4 4 4 4 2 6 3 4 2 5 6 2 6 5 4 2 2 5 3 6 6 5 5 2 6 2 5 2 2 3 3 3 2 2 3 3
## [2443] 2 2 2 6 5 3 2 2 3 2 6 2 2 3 3 6 3 2 3 3 2 3 2 2 6 3 2 6 6 2 1 5 6 6 2 2 2
## [2480] 6 2 2 2 4 2 3 2 3 2 3 2 2 6 2 5 4 5 5 4 6 2 4 4 3 2 5 2 6 4 6 6 6 1 5 1 6
## [2517] 2 2 2 6 6 6 4 6 2 5 3 3 2 5 4 4 4 4 4 5 2 4 3 3 4 5 6 6 2 4 6 5 5 6 6 2 2
## [2554] 6 6 6 5 5 3 3 2 3 2 6 2 3 6 2 5 2 2 6 2 2 2 6 3 6 5 5 6 5 5 6 2 3 3 6 2 6
## [2591] 5 4 5 3 6 1 6 3 3 5 4 4 6 6 5 6 5 3 6 2 3 2 4 3 2 6 2 6 3 2 2 3 3 3 2 6 5

```

```

## [2628] 6 4 3 4 4 4 4 6 5 5 2 2 6 5 2 4 2 2 2 5 5 2 6 6 5 2 5 2 2 3 2 3 2 3 4 3 3
## [2665] 6 3 2 5 6 3 6 2 3 3 2 3 6 3 4 2 6 6 3 6 3 2 2 3 2 4 3 6 6 3 2 6 6 2 2 6 2
## [2702] 6 2 3 3 2 5 5 6 5 2 3 2 1 2 5 4 5 6 2 2 2 2 6 3 3 2 2 2 6 2 6 2 2 6 3 3 6
## [2739] 2 5 6 6 6 6 4 6 3 2 4 4 4 5 2 3 3 6 6 3 6 6 6 6 3 5 6 5 2 5 6 2 2 2 2 4 3
## [2776] 3 3 3 2 2 2 2 6 6 4 3 2 6 5 2 6 2 6 4 6 6 6 2 2 2 5 3 2 2 3 3 3 6 2 5 3 6
## [2813] 2 5 3 2 4 2 2 4 6 2 3 2 5 2 2 2 2 4 2 2 6 6 3 5 2 6 3 3 3 6 6 6 4 3 5 5 3
## [2850] 3 6 6 3 6 2 6 4 4 4 4 6 6 4 5 3 2 3 6 3 3 2 6 2 3 6 3 6 6 6 6 3 6 6 3 6 6
## [2887] 2 2 5 2 3 5 2 2 3 6 2 3 3 3 3 4 3 4 2 2 6 6 2 6 2 6 3 3 3 3 3 2 4 2 2 6 3
## [2924] 3 2 3 3 5 2 2 2 4 3 2 3 3 4 4 2 2 6 2 3 3 2 3 3 3 5 2 6 3 4 3 6 3 3 2 6 3
## [2961] 3 2 2 6 6 2 2 3 3 2 3 3 3 2 6 6 6 6 2 6 2 2 2 2 6 2 5 6 3 2 5 5 2 2 2 6 3
## [2998] 3 5 3 4 2 6 5 2 5 5 3 2 2 5 6 2 6 5 2 4 6 2 6 5 2 4 2 6 6 6 5 5 2 5 2 2 6
## [3035] 2 4 5 4 4 3 2 2 2 6 6 2 6 6 3 6 6 6 6 4 6 2 6 6 6 6 3 6 6 6 6 3 2 5 6 6 6
## [3072] 1 6 5 6 3 2 6 2 6 2 3 5 6 2 5 3 4 3 2 2 3 3 3 3 5 6 5 6 6 3 2 2 2 6 4 3 3
## [3109] 6 6 2 6 3 6 6 6 3 2 3 3 2 6 3 6 6 6 3 6 6 6 3 4 2 3 2 1 6 2 2 3 5 2 2 1 6
## [3146] 6 6 2 2 6 6 6 6 3 6 3 6 2 5 6 2 2 6 2 2 6 2 3 6 6 3 3 2 2 3 5 2 3 3 2 5 2
## [3183] 3 4 3 6 4 3 2 3 3 2 5 6 4 6 4 2 2 6 4 5 5 6 2 2 2 4 4 2 2 5 2 6 6 4 6 4 6
## [3220] 5 6 3 2 6 6 3 6 2 2 2 2 2 4 2 6 3 4 2 6 6 6 6 6 6 6 2 2 3 6 4 2 3 3 6 2
## [3257] 3 4 4 4 3 2 3 3 2 5 6 5 2 6 5 3 3 3 4 5 3 3 2 3 4 4 4 3 4 6 3 4 2 3 4 4 3
## [3294] 6 6 3 3 4 3 2 4 2 6 6 6 6 3 6 3 5 2 3 6 2 6 2 6 3 5 3 3 6 6 2 6 2 4 2 6 2
## [3331] 4 2 6 3 3 2 3 2 5 6 6 6 3 6 4 5 3 2 6 3 6 3 6 2 6 4 4 5 4 3 2 2 2 4 4 4 3
## [3368] 6 2 3 6 3 3 2 6 3 6 6 6 6 3 6 2 6 6 2 3 2 3 3 2 6 3 2 6 2 6 3 3 3 5 3 3 3
## [3405] 2 2 3 2 3 3 2 5 2 2 6 6 2 2 6 6 2 2 4 3 5 3 4 4 3 3 6 3 6 3 4 2 2 4 2 6 2
## [3442] 4 6 2 6 3 6 4 3 3 2 6 2 3 4 3 3 3 4 3 3 2 4 2 6 3 6 6 6 3 3 3 3 6 6 6 5 4
## [3479] 2 4 2 4 4 2 2 2 3 3 2 4 3 6 6 6 4 3 3 5 2 3 2 3 2 4 3 2 3 2 6 3 5 2 6 6 2
## [3516] 3 5 3 3 3 6 2 5 2 2 2 3 3 3 4 4 2 3 5 3 4 3 5 6 4 6 4 3 3 6 3 3 6 6 4 3 6
## [3553] 3 4 3 5 5 4 5 5 2 2 4 3 3 2 3 6 3 2 3 6 6 6 4 4 6 2 3 4 4 4 4 4 3 6 3 4 2
## [3590] 2 4 2 3 3 3 3 3 3 3 6 6 6 3 3 3 3 5 6 2 2 6 6 6 5 5 2 5 6 6 3 6 4 6 3 6 5
## [3627] 3 6 4 6 6 2 3 2 5 6 6 2 5 4 2 2 2 2 5 2 6 6 3 2 6 6 2 2 6 6 6 4 3 4 2 6 2
## [3664] 3 2 6 6 2 2 3 6 3 3 3 2 2 3 2 2 2 2 2 6 6 2 6 6 6 3 3 2 6 6 6 2 6 3 6 6
## [3701] 5 4 6 6 2 4 6 3 3 6 6 6 2 3 6 5 2 2 2 6 2 3 6 2 3 3 4 6 2 6 3 3 6 2 4 6 4
## [3738] 2 2 2 2 3 5 6 3 2 6 2 2 2 2 5 5 3 3 5 2 2 5 2 5 2 2 6 6 6 4 4 4 4 4 2 3 4
## [3775] 4 2 2 3 2 6 2 6 6 2 2 5 6 5 6 2 6 6 6 5 6 5 2 3 4 6 6 3 6 3 3 6 6 2 2 6 6
## [3812] 2 2 2 6 2 2 6 6 2 6 6 3 6 3 3 3 6 6 6 6 5 6 5 6 3 3 6 6 3 3 3 3 4 6 3 6 2
## [3849] 3 4 2 3 3 3 6 2 6 4 6 4 6 5 3 6 3 3 6 6 3 2 6 2 2 2 3 4 5 4 6 5 5 4 4 4 4
## [3886] 6 6 4 2 2 6 4 2 2 6 3 4 2 2 2 2 3 2 3 6 6 6 6 2 6 6 6 5 6 6 2 6 3 5 6 3 2
## [3923] 6 3 2 2 2 2 3 4 6 4 2 4 2 4 3 3 5 6 3 2 6 5 4 3 2 6 3 4 4 6 2 5 2 6 2 4 2
## [3960] 2 2 6 3 4 6 4 3 3 6 2 4 5 5 3 6 3 5 4 3 3 6 2 5 2 3 3 3 2 5 6 6 6 6 4 3 4
## [3997] 2 2 5 2 2 4 2 6 3 2 2 4 2 4 4 4 2 3 2 4 4 2 4 2 3 4 2 4 2 3 4 3 4 3 3 4 4
## [4034] 4 6 3 6 3 6 3 6 3 4 3 4 4 6 2 3 6 6 6 6 4 3 6 6 3 6 6 5 5 3 2 3 3 6 5 2 3
## [4071] 3 2 6 5 2 6 5 4 6 2 4 4 4 3 4 2 2 2 3 2 4 3 6 6 6 2 6 6 3 6 3 2 2 3 3 4 6
## [4108] 3 6 4 4 5 6 6 1 3 4 1 4 3 6 6 3 3 6 6 2 2 6 3 3 3 6 2 6 2 6 6 2 3 6 3 2 2
## [4145] 6 3 6 4 3 6 4 2 6 6 6 3 6 6 6 2 6 2 6 6 4 5 6 4 6 6 5 5 3 3 3 4 4 2 2 3 3
## [4182] 3 3 3 3 3 5 3 6 6 6 6 6 2 3 4 2 4 3 6 3 2 2 2 1 6 4 4 1 3 4 2 6 5 6 6 6 6
## [4219] 6 6 6 6 6 5 3 6 6 4 4 2 2 3 3 3 5 3 3 2 5 5 3 5 6 3 2 5 6 6 3 3 6 5 6 4 4
## [4256] 3 6 3 5 6 3 2 5 2 2 6 6 2 3 6 6 2 1 2 2 2 6 6 2 2 6 6 6 6 2 2 3 6 2 2 2 6
## [4293] 6 2 2 3 2 6 2 5 6 6 6 4 4 4 6 3 3 3 6 3 3 3 3 6 5 3 2 3 2 6 3 6 5 2 4 5
## [4330] 3 3 6 5 6 4 2 2 6 2 6 6 3 1 1 2 6 6 3 6 3 2 6 5 1 4 4 2 2 6 6 6 6 2 6 2 2
## [4367] 6 2 6 4 6 6 2 6 6 6 2 2 2 2 6 4 4 4 6 5 6 3 3 4 3 3 2 6 5 2 6 2 6 3 3 5 2
## [4404] 5 2 6 6 3 6 2 2 2 6 5 5 2 5 6 6 2 4 6 5 3 3 3 6 6 3 6 3 5 2 2 3 3 6 5 5 2
## [4441] 2 2 5 2 5 5 5 6 4 6 5 6 2 2 6 6 2 6 2 6 2 5 5 2 2 6 2 2 2 2 5 3 2 6 5 6 3
## [4478] 6 5 5 2 6 5 2 5 2 5 1 6 6 6 4 3 5 3 4 4 5 6 2 2 6 5 2 6 3 5 5 5 3 6 2 3 5
## [4515] 5 6 2 5 2 5 3 6 6 2 3 5 6 6 2 6 5 3 6 5 5 5 3 2 2 6 2 3 2 6 6 2 2 3 6 2 2
## [4552] 2 2 5 2 5 2 2 5 2 2 2 2 6 2 2 2 6 5 6 2 6 2 2 3 2 6 6 2 2 2 4 3 1 2 2 2 2
## [4589] 3 3 6 6 2 2 2 2 6 2 2 2 6 6 2 2 6 3 3 3 4 4 2 2 2 5 2 2 6 2 2 1 5 6 5 2 6

```

```

## [4626] 2 6 6 6 6 2 5 3 6 5 3 3 2 3 6 3 3 2 6 2 2 3 6 3 4 3 3 2 5 2 5 2 4 5 3 3 2
## [4663] 3 3 6 3 6 2 5 2 5 3 6 6 5 2 4 2 2 5 3 2 2 5 5 2 3 5 5 5 3 6 2 1 1 2 6 6 2
## [4700] 6 5 2 2 6 6 6 3 6 3 2 2 6 2 2 2 5 2 6 1 4 6 2 5 2 6 6 2 2 2 6 4 2 2 2 4 2
## [4737] 6 2 6 4 1 6 2 6 6 6 6 6 6 5 6 4 5 2 6 2 6 2 6 5 2 6 2 3 2 6 2 5 2 6 5 2 6
## [4774] 6 6 5 6 2 2 5 2 6 5 5 2 5 6 6 6 2 6 2 6 5 2 6 6 6 2 6 6 2 6 3 6 2 2 3 6 2
## [4811] 6 6 3 5 5 2 2 2 2 5 5 2 6 6 5 6 3 4 2 6 2 5 2 6 6 6 6 2 2 2 2 2 2 5 2 2 6
## [4848] 6 6 6 3 2 2 4 3 3 3 3 3 3 2 3 2 4 3 2 6 2 4 2 2 2 2 6 2 6 2 4 6 2 2 6 3 2
## [4885] 2 2 2 3 3 5 5 3 5 2 3 3 3 2 2 5 5 2 2 6 2 6 4 2 2 6 6 2 5 4 5 2 2 2 6 2 5
## [4922] 2 2 2 6 3 5 2 2 3 3 6 6 5 3 3 3 1 2 2 2 2 3 3 3 3 6 1 2 6 2 5 5 6 2 6 2 5
## [4959] 5 5 2 6 2 5 5 2 2 3 2 2 2 6 2 6 6 3 3 6 4 2 6 2 6 3 6 6 4 5 5 2 2 2 2 3 3
## [4996] 6 3 3 2 6 2 6 2 5 2 2 3 3 2 6 6 3 3 6 3 4 5 4 3 6 5 2 6 2 6 6 3 2 6 6 6 2
## [5033] 5 5 5 2 2 4 2 2 2 2 2 4 6 3 2 6 6 2 2 6 5 2 5 3 6 2 6 2 3 2 2 3 2 2 4 2 6
## [5070] 3 2 6 2 3 3 5 6 3 2 2 6 2 5 5 2 2 3 5 5 6 2 6 6 6 2 6 3 6 2 6 6 3 2 6 2 6
## [5107] 2 3 3 3 5 2 5 2 5 3 2 6 6 4 6 3 4 6 6 6 6 5 6 3 3 5 6 3 3 2 2 6 5 2 6 6 3
## [5144] 3 4 6 4 6 3 2 2 2 2 5 2 2 5 2 1 5 5 3 5 5 5 2 5 2 5 5 1 2 5 2 2 6 2 2 5 6
## [5181] 6 5 2 5 2 3 2 2 2 2 3 3 3 2 2 2 6 3 4 5 6 6 2 5 3 6 5 6 6 3 5 2 6 6 2 3 2
## [5218] 2 2 3 2 2 2 5 3 2 3 3 3 2 2 6 2 3 2 5 2 5 2 6 6 6 6 6 6 5 2 6 6 2 3 2 3 2
## [5255] 6 3 6 6 6 2 6 5 6 6 6 6 2 5 2 3 2 5 2 2 3 2 3 2 2 3 2 2 3 2 3 3 2 3 2 6 2
## [5292] 2 2 2 6 2 3 2 3 6 6 3 3 3 3 5 2 3 6 1 3 3 3 2 3 3 2 6 4 6 6 6 6 2 2 2 6 5
## [5329] 2 3 3 2 2 2 5 5 6 6 6 6 6 6 6 6 2 3 6 6 6 2 3 3 6 6 2 3 3 6 2 2 1 5 6 3
## [5366] 3 3 6 3 6 6 6 2 4 2 6 2 2 2 4 2 2 3 6 2 3 4 4 3 3 3 6 3 2 6 2 5 6 3 1 5 6
## [5403] 2 1 5 5 2 2 2 3 6 6 3 6 2 6 6 2 2 6 6 4 3 2 5 2 2 6 2 2 6 6 2 6 2 2 2 6 2
## [5440] 5 3 2 2 5 3 2 3 5 5 2 2 2 2 2 6 2 2 6 4 4 3 3 3 2 2 2 4 3 3 6 3 3 3 3 2 6
## [5477] 2 3 5 2 2 6 6 2 5 6 5 2 6 2 6 6 5 6 5 3 3 2 6 5 1 5 5 2 5 5 5 6 6 6 2 5 2
## [5514] 6 2 6 4 2 2 6 3 5 2 2 2 2 2 3 3 2 2 2 5 5 3 2 6 2 3 6 6 2 2 3 3 6 6 2 6 6
## [5551] 3 2 2 3 2 6 5 2 2 3 3 2 2 3 2 2 3 3 2 6 2 2 3 3 6 2 2 3 3 6 3 6 2 2 2 2 4
## [5588] 6 2 4 2 5 2 2 2 2 6 2 2 2 2 2 2 2 2 1 2 2 6 2 6 2 3 3 2 6 2 3 2 5 2 6 6 6
## [5625] 2 6 3 5 6 2 2 2 2 2 3 3 3 2 2 3 6 6 6 6 6 6 2 2 2 6 2 6 6 6 2 6 2 2 2 5 5
## [5662] 2 6 5 2 3 3 3 6 3 5 3 2 5 6 6 2 2 2 2 2 6 3 2 5 2 2 5 6 3 5 2 2 2 2 6 5 6
## [5699] 2 2 3 3 5 5 2 6 6 2 6 3 5 2 2 2 2 6 6 6 6 6 6 3 5 5 6 6 4 4 5 2 4 4 2 5 6
## [5736] 2 4 5 6 6 6 6 6 6 3 2 6 2 5 3 3 2 3 3 3 3 3 3 3 3 3 3 2 2 5 6 2 2 6 3 2 3
## [5773] 2 3 6 3 2 6 4 2 5 3 5 5 4 5 2 2 2 5 6 2 2 2 5 5 6 6 2 2 5 2 3 6 5 6 2 5 2
## [5810] 6 3 6 5 4 4 4 2 3 4 2 6 2 1 5 3 3 2 3 6 2 5 5 2 2 2 2 5 2 2 6 6 6 5 2 2 2
## [5847] 2 2 3 2 2 2 5 2 2 2 2 2 2 6 2 3 6 2 6 2 2 6 3 3 4 2 2 2 2 2 3 1 6 2 2 2 2
## [5884] 2 2 2 2 5 2 2 3 2 3 2 3 2 2 2 6 3 6 3 2 2 2 6 2 5 2 6 2 2 5 2 2 2 6 2 2 3
## [5921] 6 2 6 2 3 3 3 6 3 3 3 6 6 6 6 6 6 2 6 6 6 2 2 2 2 6 6 3 5 6 6 3 3 6 3 3
## [5958] 3 4 6 2 2 6 2 6 2 6 2 2 2 5 2 2 3 3 6 5 2 6 6 6 2 5 3 3 3 2 2 2 3 4 3 6 6
## [5995] 6 3 3 3 6 3 6 3 3 3 3 6 5 3 6 3 5 2 5 2 6 3 2 2 2 2 2 2 2 2 2 2 4 3 3 6 5
## [6032] 2 3 6 3 5 3 6 2 6 2 6 2 2 2 2 2 2 6 6 3 3 2 6 3 3 3 2 2 6 6 2 2 6 6 6 6 2
## [6069] 2 2 5 5 6 6 2 2 3 3 2 6 3 6 2 2 5 5 5 2 2 2 2 5 3 5 3 5 6 2 2 3 3 6 2 6 3
## [6106] 2 5 6 2 6 2 2 2 4 2 5 5 1 3 4 3 1 3 6 3 4 5 2 3 3 3 2 6 2 3 6 5 6 5 2 2 5
## [6143] 5 2 2 2 1 3 5 2 2 2 2 2 2 6 4 6 2 2 5 6 2 2 6 3 6 6 2 5 2 2 2 2 6 5 2 2 6
## [6180] 2 6 6 6 2 6 6 2 6 6 3 4 3 2 6 2 2 5 2 5 2 2 6 6 6 2 2 3 5 1 5 5 6 6 3 6 2
## [6217] 6 6 3 2 2 6 2 5 6 3 2 2 5 6 2 3 3 6 2 3 2 3 3 2 6 2 3 2 6 5 2 4 5 5 2 4 5
## [6254] 6 6 6 6 6 5 5 6 2 2 2 2 4 2 2 2 6 3 3 2 6 6 6 2 6 3 6 1 5 2 3 2 6 6 6 6 6
## [6291] 3 3 6 3 2 5 2 2 3 3 6 6 5 4 2 2 2 2 6 2 2 2 2 2 5 5 6 6 5 2 2 6 3 2 2 2 6
## [6328] 2 5 2 6 6 6 5 5 5 6 2 5 3 6 6 6 2 4 6 2 3 3 6 3 5 2 2 2 2 2 6 6 6 2 5 6 2
## [6365] 2 6 3 3 3 3 3 4 5 2 6 6 2 2 2 6 6 6 6 3 5 6 2 3 2 6 6 6 3 3 6 6 2 2 2 6 6
## [6402] 2 2 5 2 6 2 2 6 6 5 2 3 2 2 6 2 5 2 3 6 3 6 6 6 6 5 6 5 6 6 6 2 2 2 5 4 2
## [6439] 1 6 3 2 3 3 5 6 2 3 3 3 3 2 5 5 3 3 2 3 2 2 2 5 2 6 2 2 5 6 5 6 2 3 2 2 2
## [6476] 2 5 5 3 3 2 6 6 3 3 2 5 6 2 2 2 2 2 3 2 2 2

```

```
##
```

```
## Within cluster sum of squares by cluster:
```

```
## [1] 198652.2 364222.2 440016.8 400015.4 227569.8 416158.5
```

```
## (between_SS / total_SS = 91.1 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

3. Results of the T-test on the sample means of the lowest and highest clusters accross all variables.

```
wine_data_numeric$cluster <- as.factor(kmeans_result$cluster)

wine_data_numeric %>%
  filter(cluster %in% c(4, 6)) %>%
  select(-cluster) %>%
  summarise(across(everything(),
    ~ {
      test_result <- t.test(. ~ wine_data_numeric$cluster[wine_data_numeric$cluster %in%
        c(t_stat = test_result$statistic,
          p_value = test_result$p.value)
      },
      .names = "{col}_t_test")) %>%
  pivot_longer(everything(), names_to = "Variable",
    values_to = "Test_Result") %>%
  mutate(Statistic = rep(c("T Statistic", "P Value"),
    each = ncol(wine_data_numeric) - 1)) %>%
  pivot_wider(names_from = Statistic, values_from = Test_Result) %>%
  separate(Variable, into = c("Variable", "Type"), sep = "_") %>%
  mutate(`High Quality Wine has ...` = case_when(
    `T Statistic` > 0 ~ "Lower Level",
    `T Statistic` <= 0 ~ "Higher Level")) %>%
  select(-Type) -> table_t
```

```
## Warning: Returning more (or less) than 1 row per 'summarise()' group was deprecated in
## dplyr 1.1.0.
## i Please use 'reframe()' instead.
## i When switching from 'summarise()' to 'reframe()', remember that 'reframe()'
## always returns an ungrouped data frame and adjust accordingly.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
## Warning: Expected 2 pieces. Additional pieces discarded in 12 rows [1, 2, 3, 4, 5, 6, 7,
## 8, 9, 10, 11, 12].
```

Research Question #2: Can we predict the quality of a wine given its characteristic?


```
levels(wine_data$quality)
```

```
## [1] "3" "4" "5" "6" "7" "8" "9"
```

Method #1: KNN

1. Splitting the data into a testing and training dataset.

```
set.seed(1)

n <- nrow(wine_data)

z <- sample(n, n/2)

wine_test <- wine_data[z,]

wine_train <- wine_data[-z,][1:(n/2),]

x.train <- wine_train[,2:12]
x.test <- wine_test[,2:12]

y.train <- wine_train$quality
y.test <- wine_test$quality

dim(x.train)[1] == length(y.train) # Test to see if length is the same. Should be TRUE
```

```
## [1] TRUE
```

```
dim(x.test)[1] == length(y.test) # Test to see if length is the same. Should be TRUE
```

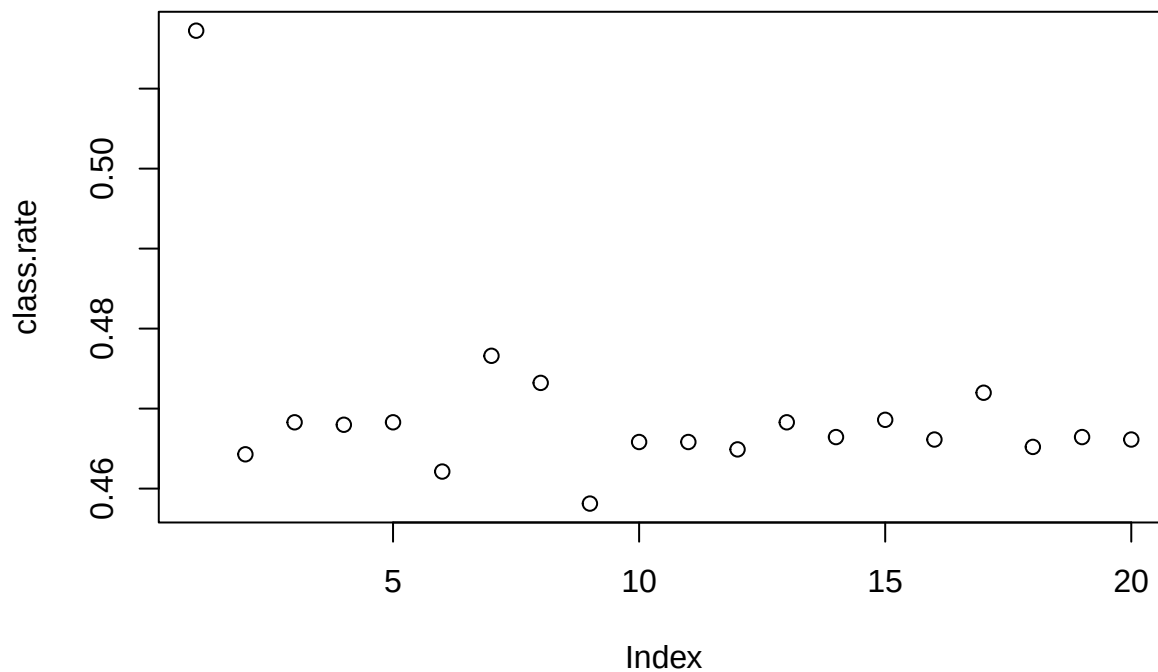
```
## [1] TRUE
```

2. Determining the optimal number of nearest neighbors “K”

```
class.rate = rep(0,20)

for (K in 1:20) {
  knn.result = knn(x.train, x.test, y.train, K)
  class.rate[K] = mean( y.test == knn.result )
}

plot(class.rate)
```



```
max(class.rate) # optimal number of K neighbors is 1
```

```
## [1] 0.5172414
```

3. Cross Validation - CClassification Rate and Confusion table using Optimal number of nearest Neighbors.

```
knn_cv_wine_quality <- knn(x.train, x.test, y.train, 1)
```

```
table(y.test, knn_cv_wine_quality)
```

```
##      knn_cv_wine_quality
## y.test  3  4  5  6  7  8  9
##      3  0  2  7  2  3  0  0
##      4  4 12 35 41  8  1  0
##      5  5 29 55 73 92  9  0
##      6  2 40 32 84 17 40  0
##      7  1  3 65 21 23 26  1
##      8  0  5  8 30 26 29  0
##      9  0  0  0  1  1  0  0
```

```
knn_cv_classification_rate <- mean(y.test == knn_cv_wine_quality)
```

```
mad_knn <- mean(abs(as.numeric(wine_test$quality) - as.numeric(knn_cv_wine_quality)))
```

Method #2: Classification Trees

1. Classification Tree for Quality

```
tree_wine_quality <- tree(quality ~ ., data = wine_train)
```

```
## Warning in tree(quality ~ ., data = wine_train): NAs introduced by coercion
```

```
tree_wine_quality
```

```
## node), split, n, deviance, yval, (yprob)
##      * denotes terminal node
##
##  1) root 3248 8303.0 6 ( 0.0049261 0.0354064 0.3303571 0.4356527 0.1634852 0.0292488 0.0009236 )
##    2) alcohol < 10.625 1947 4304.0 5 ( 0.0061633 0.0405752 0.4694402 0.4083205 0.0662558 0.0092450 0
##      4) volatile.acidity < 0.2625 799 1812.0 6 ( 0.0050063 0.0225282 0.2941176 0.5394243 0.1201502 0
##      5) volatile.acidity > 0.2625 1148 2257.0 5 ( 0.0069686 0.0531359 0.5914634 0.3170732 0.0287456 0
##        10) alcohol < 9.85 782 1286.0 5 ( 0.0051151 0.0460358 0.6764706 0.2647059 0.0076726 0.0000000 0
##        11) alcohol > 9.85 366 873.3 6 ( 0.0109290 0.0683060 0.4098361 0.4289617 0.0737705 0.0081967 0
##    3) alcohol > 10.625 1301 3308.0 6 ( 0.0030746 0.0276710 0.1222137 0.4765565 0.3089931 0.0591852 0
##      6) alcohol < 11.775 728 1834.0 6 ( 0.0027473 0.0384615 0.1771978 0.5027473 0.2417582 0.0370879 0
##      7) alcohol > 11.775 573 1377.0 6 ( 0.0034904 0.0139616 0.0523560 0.4432810 0.3944154 0.0872600 0
```

```
summary(tree_wine_quality)
```

```
##
## Classification tree:
## tree(formula = quality ~ ., data = wine_train)
## Variables actually used in tree construction:
## [1] "alcohol"          "volatile.acidity"
## Number of terminal nodes: 5
## Residual mean deviance: 2.215 = 7182 / 3243
## Misclassification error rate: 0.4652 = 1511 / 3248
```

```
# not all class are represented as terminal node classifications (3, 4 & 8 are not represented).
```

2. Calculating the Classification Rate for Unpruned Tree

```
tree_hat <- predict(tree_wine_quality, wine_test, type = "class")
```

```
## Warning in pred1.tree(object, tree.matrix(newdata)): NAs introduced by coercion
```

```
table(tree_hat, wine_test$quality)
```

```
##
## tree_hat    3    4    5    6    7    8    9
##           3    0    0    0    0    0    0
```

```
##      4      0      0      0      0      0      0      0
##      5      3     43    483    234     11      1      0
##      6     11     58    582   1186    537     97      2
##      7      0      0      0      0      0      0      0
##      8      0      0      0      0      0      0      0
##      9      0      0      0      0      0      0      0
```

```
tree_classification_rate <- mean(tree_hat == wine_test$quality)
```

3. Pruning the Classification tree on misclassification rate.

The optimal number of terminal nodes on a tree is 8. The same number of terminal nodes as the tree above

```
set.seed(1)
```

```
tree_cv_wine_quality <- cv.tree( tree_wine_quality, FUN = prune.misclass )
```

```
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion
```

```
## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion
```

```
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion
```

```
## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion
```

```
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion
```

```
## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion
```

```
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion
```

```
## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion
```

```
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion
```

```
## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion
```

```
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion
```

```
## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion
```

```
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion
```

```
## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion

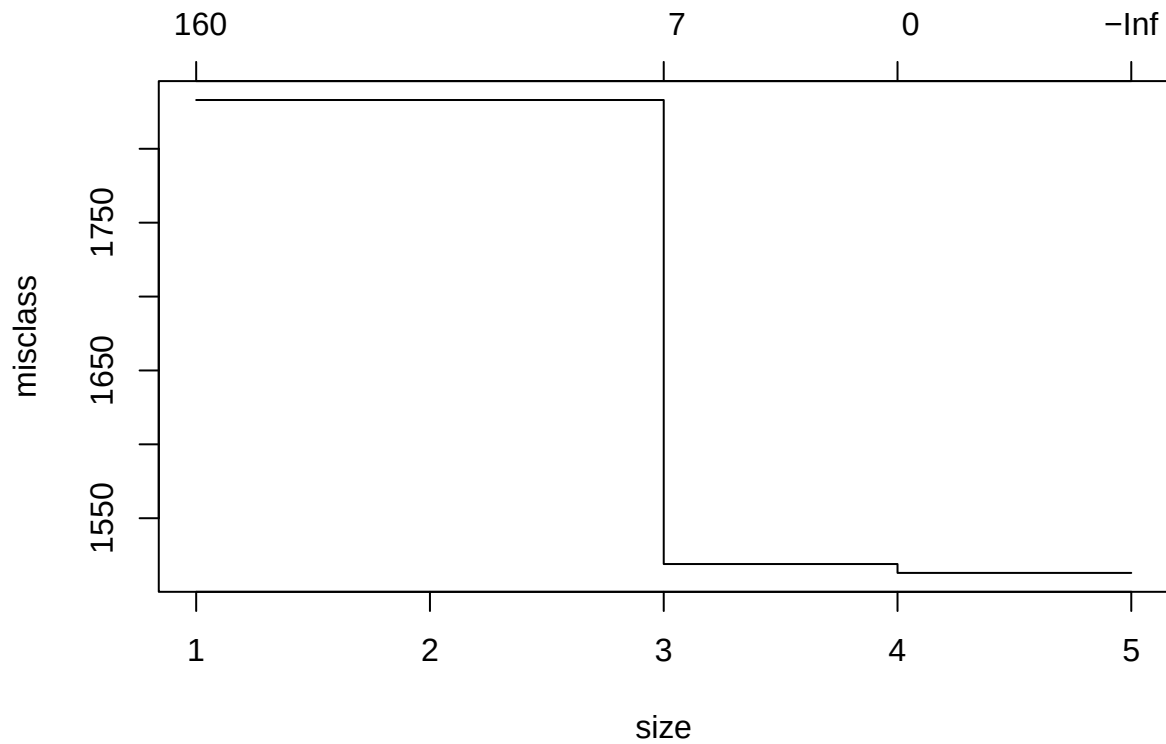
## Warning in tree(model = m[rand != i, , drop = FALSE]): NAs introduced by
## coercion

## Warning in pred1.tree(tree, tree.matrix(nd)): NAs introduced by coercion
```

```
tree_cv_wine_quality
```

```
## $size
## [1] 5 4 3 1
##
## $dev
## [1] 1513 1513 1519 1833
##
## $k
## [1] -Inf 0.0 7.0 157.5
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune" "tree.sequence"
```

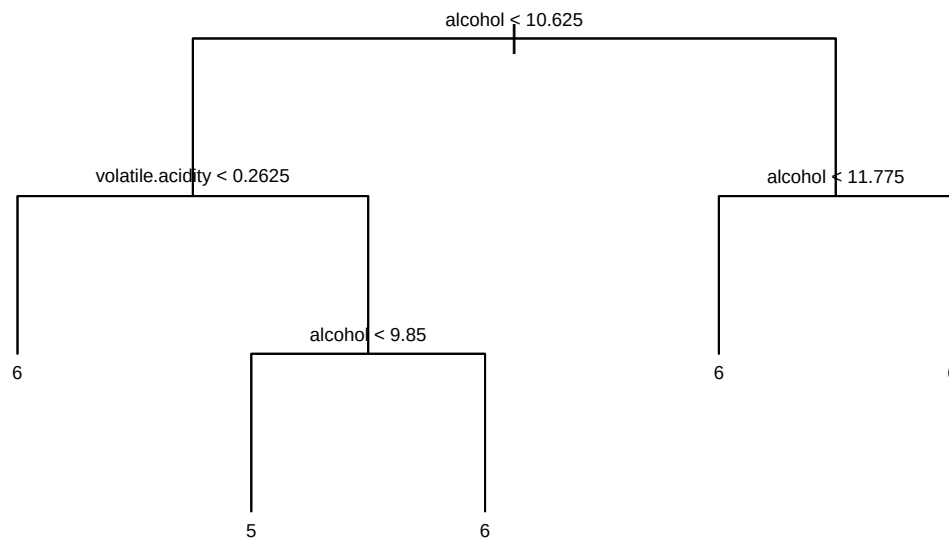
```
plot(tree_cv_wine_quality)
```



```
tree_pruned_wine_quality <- prune.misclass(tree_wine_quality, best = 5)
```

4. Plotting Classification Tree

```
plot(tree_pruned_wine_quality, type = "uniform")
text(tree_pruned_wine_quality, cex = 0.6)
```



of the Classification Tree

5. Cross Validation

```
tree_hat <- predict(tree_pruned_wine_quality, wine_test, type = "class")
```

```
## Warning in pred1.tree(object, tree.matrix(newdata)): NAs introduced by coercion
```

```
table(tree_hat, wine_test$quality)
```

```
##
## tree_hat      3      4      5      6      7      8      9
##           3      0      0      0      0      0      0      0
##           4      0      0      0      0      0      0      0
##           5      3     43    483    234     11      1      0
##           6     11     58    582   1186    537     97      2
##           7      0      0      0      0      0      0      0
##           8      0      0      0      0      0      0      0
##           9      0      0      0      0      0      0      0
```

```
tree_pruned_classification_rate <- mean(tree_hat == wine_test$quality)
```

```
mad_tree <- mean(abs(as.numeric(wine_test$quality) - as.numeric(tree_hat)))
```

```
##Method #3: Random Forest
```

1. Creaing the Traing Random Forest

```
set.seed(1)
```

```
forest_wine_quality <- randomForest(quality ~ ., data = wine_train)
```

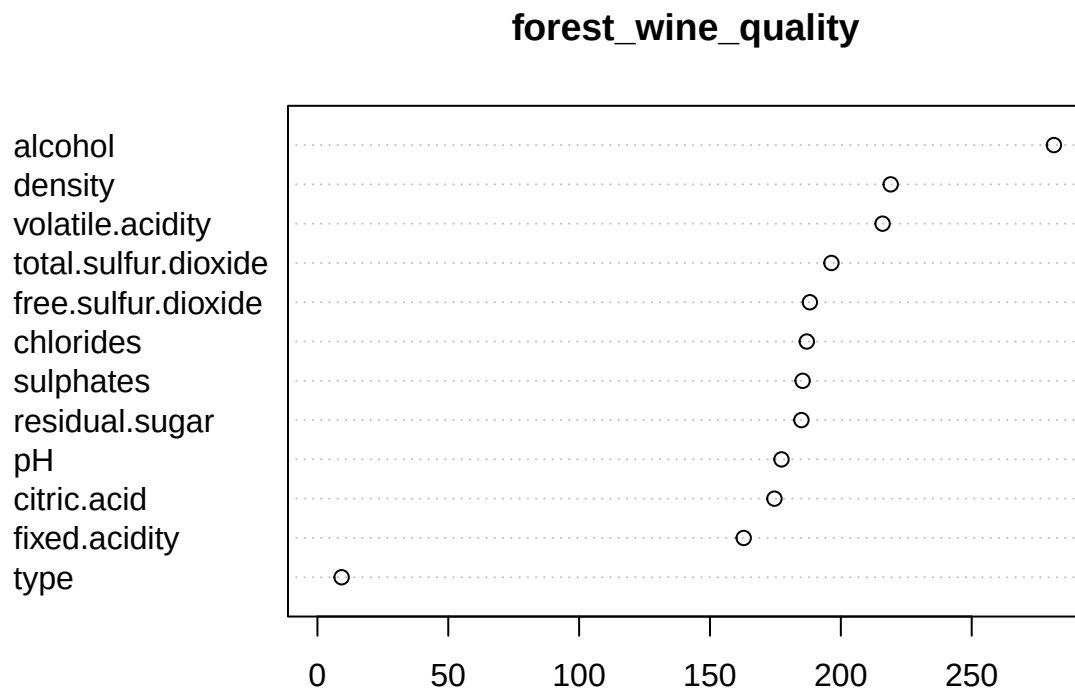
```
forest_wine_quality
```

```
##
## Call:
## randomForest(formula = quality ~ ., data = wine_train)
##           Type of random forest: classification
##           Number of trees: 500
## No. of variables tried at each split: 3
##
##           OOB estimate of  error rate: 35.04%
## Confusion matrix:
##   3 4  5   6   7  8 9 class.error
## 3 0 2   9   5   0 0 0  1.0000000
## 4 1 8  55  50   1 0 0  0.9304348
## 5 0 0 732 334   7 0 0  0.3178006
## 6 0 3 235 1080 96   1 0  0.2367491
## 7 0 0  19  240 269   3 0  0.4934087
## 8 0 0   1   40  33 21 0  0.7789474
## 9 0 0   0    1   2  0 0  1.0000000
```

```
importance(forest_wine_quality)
```

```
##               MeanDecreaseGini
## fixed.acidity      162.881103
## volatile.acidity   215.943020
## citric.acid        174.611373
## residual.sugar     184.942953
## chlorides          186.973404
## free.sulfur.dioxide 188.164176
## total.sulfur.dioxide 196.405281
## density            219.058017
## pH                 177.318223
## sulphates          185.396477
## alcohol            281.417878
## type               9.240769
```

```
varImpPlot(forest_wine_quality)
```



MeanDecreaseGini

2.

Searching for the optimal number of trees and number of variables considered at each branch.

```
set.seed(1)

err.rate <- numeric(10) # To store error rates for each mtry
n.trees <- numeric(10)  # To store the number of optimal trees for each mtry

for (m in 1:10) {
  forest_wine_quality_m <- randomForest(quality ~ ., data = wine_train,
                                         mtry = m )
  opt.tree <- which.min(forest_wine_quality_m$err.rate)
```



```

forest_wine_quality_m <- randomForest(quality ~ ., data = wine_train,
                                     mtry = m, ntree = opt.tree)
yhat_rf_m <- predict(forest_wine_quality_m, newdata = wine_test, type = "class")
err.rate[m] <- 1 - mean(wine_test$quality == yhat_rf_m)
n.trees[m] <- opt.tree
}

```

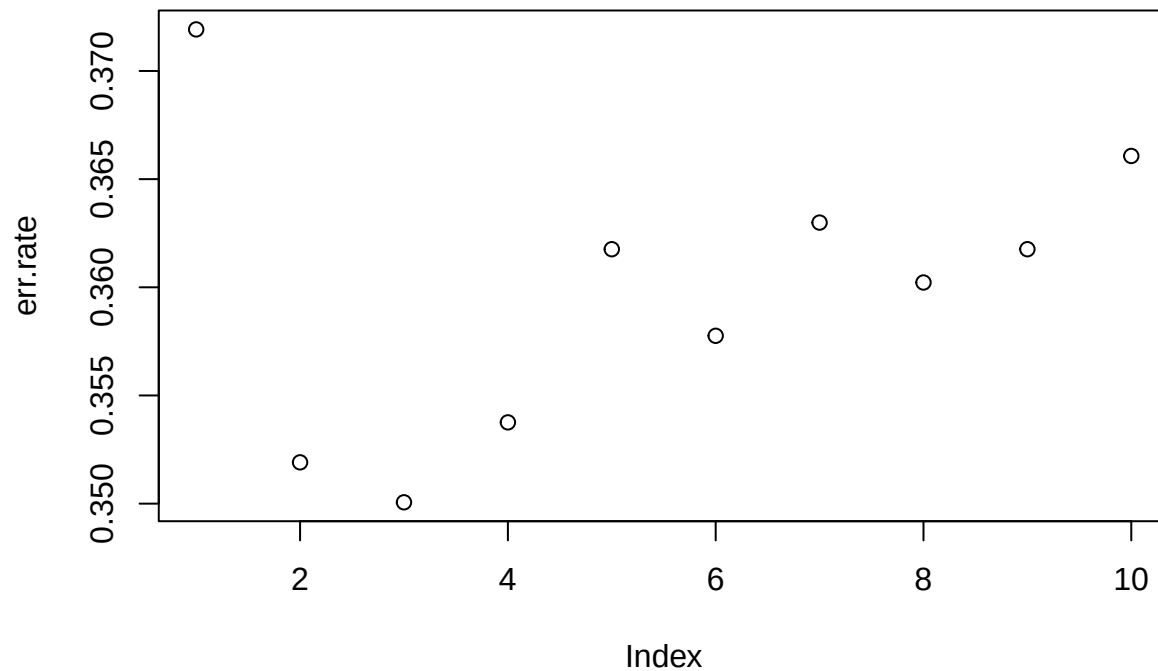
```
err.rate
```

```
## [1] 0.3719212 0.3519089 0.3500616 0.3537562 0.3617611 0.3577586 0.3629926
## [8] 0.3602217 0.3617611 0.3660714
```

```
n.trees
```

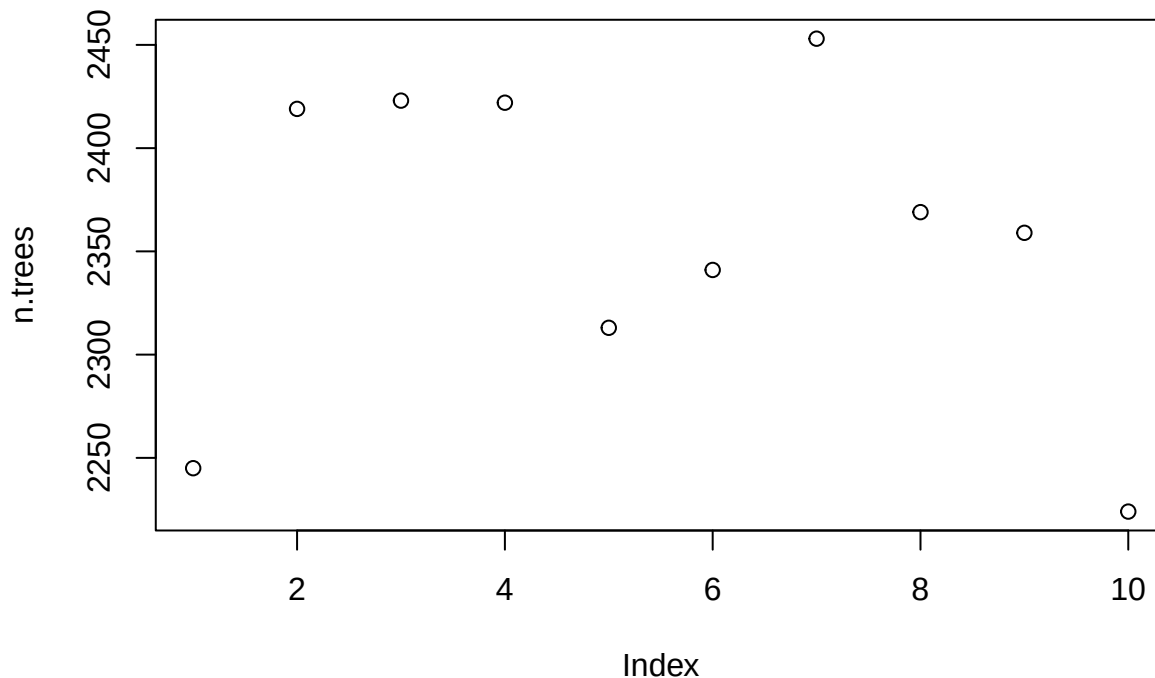
```
## [1] 2245 2419 2423 2422 2313 2341 2453 2369 2359 2224
```

```
plot(err.rate); line(err.rate)
```



```
##
## Call:
## line(err.rate)
##
## Coefficients:
## [1] 0.348471 0.001591
```

```
plot(n.trees); line(n.trees)
```



```
##
## Call:
## line(n.trees)
##
## Coefficients:
## [1] 2440.792 -9.417
```

optimal solution is 2192 trees and 2 variables considered at each branch.

3. Cross Validation with Optimal number of trees and number of variables considered at each branch.

```
set.seed(1)

forest_cv_wine_quality <- randomForest(quality ~ ., data = wine_train,
                                         mtry = 3, ntree = 2423 )

yhat_rf <- predict(forest_cv_wine_quality, newdata = wine_test, type = "class")

table(wine_test$quality, yhat_rf)
```

```
##      yhat_rf
##      3      4      5      6      7      8      9
## 3      0      0      3     10      1      0      0
## 4      0      7     61     32      1      0      0
## 5      0      2    733    322      8      0      0
## 6      0      1    250   1090     78      1      0
## 7      0      0     14    282    250      2      0
## 8      0      0      0     45     25     28      0
## 9      0      0      0      2      0      0      0
```

```
forest_classification_rate <- mean(wine_test$quality == yhat_rf)

mad_forest <- mean(abs(as.numeric(wine_test$quality) - as.numeric(yhat_rf)))
```

Method #4: Support Vecot Machines

1. Creating a SVM model with a linear bounder

```
SVM_wine_quality <- svm(quality ~ ., data = wine_train, kernel="linear")

summary(SVM_wine_quality)
```

```
##
## Call:
## svm(formula = quality ~ ., data = wine_train, kernel = "linear")
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##         cost:  1
##
## Number of Support Vectors:  3017
##
##   ( 915 531 115 1342 95 16 3 )
##
##
## Number of Classes:  7
##
## Levels:
##   3 4 5 6 7 8 9
```

2. Tuning the SVM model to detiermen the optimal kernel and cost.

```
set.seed(1)

svm_tuned <- tune(
  svm,
  quality ~ .,
  data = wine_train,
  ranges = list(
    cost = 10^seq(-2, 2, by = 1),
    kernel = c("linear", "polynomial", "radial", "sigmoid") # Test multiple kernels
  )
)
```

```
summary(svm_tuned)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##   cost kernel
##     10 radial
##
## - best performance: 0.4220921
##
## - Detailed performance results:
##   cost      kernel      error dispersion
## 1  1e-02      linear 0.4682887 0.01900000
## 2  1e-01      linear 0.4642830 0.02078634
## 3  1e+00      linear 0.4627455 0.02002080
## 4  1e+01      linear 0.4636686 0.01985583
## 5  1e+02      linear 0.4633609 0.01938088
## 6  1e-02 polynomial 0.5584834 0.02513259
## 7  1e-01 polynomial 0.5190826 0.02450784
## 8  1e+00 polynomial 0.4565717 0.02599846
## 9  1e+01 polynomial 0.4390209 0.03113474
## 10 1e+02 polynomial 0.4528851 0.02800888
## 11 1e-02      radial 0.5643324 0.02489330
## 12 1e-01      radial 0.4673542 0.02150864
## 13 1e+00      radial 0.4297930 0.02438804
## 14 1e+01      radial 0.4220921 0.03318718
## 15 1e+02      radial 0.4270237 0.02437081
## 16 1e-02      sigmoid 0.5341605 0.03003334
## 17 1e-01      sigmoid 0.4772137 0.02291533
## 18 1e+00      sigmoid 0.5584976 0.02326672
## 19 1e+01      sigmoid 0.5652887 0.03971605
## 20 1e+02      sigmoid 0.5757540 0.02641117
```

```
# optimal cost is 10 with a radial boundary.
```

3. Cross Validation with Optimal Cost and Kernel

```
SVM_wine_quality_optimal <- svm(quality ~ .,
                                data = wine_train,
                                kernel="radial",
                                cost = 10)

summary(SVM_wine_quality_optimal)
```

```
##
## Call:
## svm(formula = quality ~ ., data = wine_train, kernel = "radial",
##      cost = 10)
##
##
```

```
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: radial
##       cost:  10
##
## Number of Support Vectors:  2793
##
## ( 843 503 114 1219 95 16 3 )
##
##
## Number of Classes:  7
##
## Levels:
##  3 4 5 6 7 8 9
```

```
svm_yhat <- predict(SVM_wine_quality_optimal, data = wine_test)

length(svm_yhat)
```

```
## [1] 3248
```

```
length(wine_test$quality)
```

```
## [1] 3248
```

```
table(svm_yhat, wine_test$quality)
```

```
##
## svm_yhat   3   4   5   6   7   8   9
##      3   0   0   1   7   1   0   0
##      4   0   1  12  17   6   4   0
##      5   5  34 352 469 167  27   2
##      6   7  53 602 788 316  58   0
##      7   2  13  95 130  55   9   0
##      8   0   0   3   9   3   0   0
##      9   0   0   0   0   0   0   0
```

```
svm_cv_classification_rate <- mean(svm_yhat == wine_test$quality)

mad_svm <- mean(abs(as.numeric(wine_test$quality) - as.numeric(svm_yhat)))
```

Method #5: Artificial Neural Network

1. Searching for the optimal size of hidden nodes

```
set.seed(1)

err.rate <- numeric(10)
```

```

for (m in 1:12) {
  ann_wine_quality <- nnet(quality ~ .,
                           data = wine_train,
                           size = m)
  yhat_ann <- predict(ann_wine_quality, newdata = wine_test, type = "class")
  err.rate[m] <- 1 - mean(wine_test$quality == yhat_ann)
}

```

```

## # weights: 27
## initial value 5212.598328
## iter 10 value 4150.169951
## iter 20 value 4120.599867
## iter 30 value 4086.774995
## iter 40 value 3870.792464
## iter 50 value 3610.285149
## iter 60 value 3553.767130
## iter 70 value 3538.673368
## iter 80 value 3533.191430
## iter 90 value 3532.014817
## iter 100 value 3529.961545
## final value 3529.961545
## stopped after 100 iterations
## # weights: 47
## initial value 6170.810951
## iter 10 value 4216.777275
## iter 20 value 4151.547248
## iter 30 value 4144.425982
## iter 40 value 4129.732266
## iter 50 value 4114.689192
## iter 60 value 4022.250327
## iter 70 value 3922.511207
## iter 80 value 3705.584964
## iter 90 value 3615.116206
## iter 100 value 3556.039910
## final value 3556.039910
## stopped after 100 iterations
## # weights: 67
## initial value 7919.988519
## iter 10 value 4281.127946
## iter 20 value 4133.682121
## iter 30 value 4115.764681
## iter 40 value 4031.892416
## iter 50 value 3795.157538
## iter 60 value 3630.799628
## iter 70 value 3568.732963
## iter 80 value 3541.507030
## iter 90 value 3532.256106
## iter 100 value 3530.109504
## final value 3530.109504
## stopped after 100 iterations
## # weights: 87
## initial value 7101.790805
## iter 10 value 4204.539886

```

```

## iter 20 value 4078.260931
## iter 30 value 4064.607656
## iter 40 value 4019.862165
## iter 50 value 3861.077546
## iter 60 value 3772.640340
## iter 70 value 3731.624402
## iter 80 value 3668.072939
## iter 90 value 3649.279076
## iter 100 value 3644.071019
## final value 3644.071019
## stopped after 100 iterations
## # weights: 107
## initial value 7571.813738
## iter 10 value 4217.877827
## iter 20 value 4143.272971
## iter 30 value 4113.802485
## iter 40 value 4075.546542
## iter 50 value 3980.018813
## iter 60 value 3718.894025
## iter 70 value 3593.044625
## iter 80 value 3541.231287
## iter 90 value 3518.279148
## iter 100 value 3510.587026
## final value 3510.587026
## stopped after 100 iterations
## # weights: 127
## initial value 8196.923617
## iter 10 value 4126.436335
## iter 20 value 4026.817717
## iter 30 value 3887.009597
## iter 40 value 3805.152903
## iter 50 value 3745.174365
## iter 60 value 3699.297554
## iter 70 value 3675.390267
## iter 80 value 3648.547586
## iter 90 value 3625.974792
## iter 100 value 3611.142809
## final value 3611.142809
## stopped after 100 iterations
## # weights: 147
## initial value 6681.675099
## iter 10 value 4207.476004
## iter 20 value 4105.144017
## iter 30 value 3834.260661
## iter 40 value 3675.354781
## iter 50 value 3595.773847
## iter 60 value 3533.196296
## iter 70 value 3501.527209
## iter 80 value 3492.440875
## iter 90 value 3486.178729
## iter 100 value 3479.750396
## final value 3479.750396
## stopped after 100 iterations
## # weights: 167

```

```

## initial value 9253.504123
## iter 10 value 4136.575882
## iter 20 value 4029.660111
## iter 30 value 3981.009773
## iter 40 value 3933.160868
## iter 50 value 3830.559118
## iter 60 value 3616.991987
## iter 70 value 3500.847400
## iter 80 value 3455.068800
## iter 90 value 3418.313539
## iter 100 value 3405.785785
## final value 3405.785785
## stopped after 100 iterations
## # weights: 187
## initial value 7161.848312
## iter 10 value 4225.295899
## iter 20 value 4164.000369
## iter 30 value 4113.715935
## iter 40 value 3960.059351
## iter 50 value 3782.323093
## iter 60 value 3682.893079
## iter 70 value 3566.664441
## iter 80 value 3534.444564
## iter 90 value 3518.960091
## iter 100 value 3484.804983
## final value 3484.804983
## stopped after 100 iterations
## # weights: 207
## initial value 6040.084600
## iter 10 value 4193.819211
## iter 20 value 4124.975346
## iter 30 value 4094.505594
## iter 40 value 4031.836069
## iter 50 value 3900.086895
## iter 60 value 3715.151340
## iter 70 value 3571.334502
## iter 80 value 3519.323584
## iter 90 value 3429.999422
## iter 100 value 3389.789339
## final value 3389.789339
## stopped after 100 iterations
## # weights: 227
## initial value 9953.715028
## iter 10 value 4251.952035
## iter 20 value 4143.306805
## iter 30 value 4126.530123
## iter 40 value 4004.410349
## iter 50 value 3926.009488
## iter 60 value 3841.577308
## iter 70 value 3675.099237
## iter 80 value 3541.185587
## iter 90 value 3497.579560
## iter 100 value 3480.180268
## final value 3480.180268

```

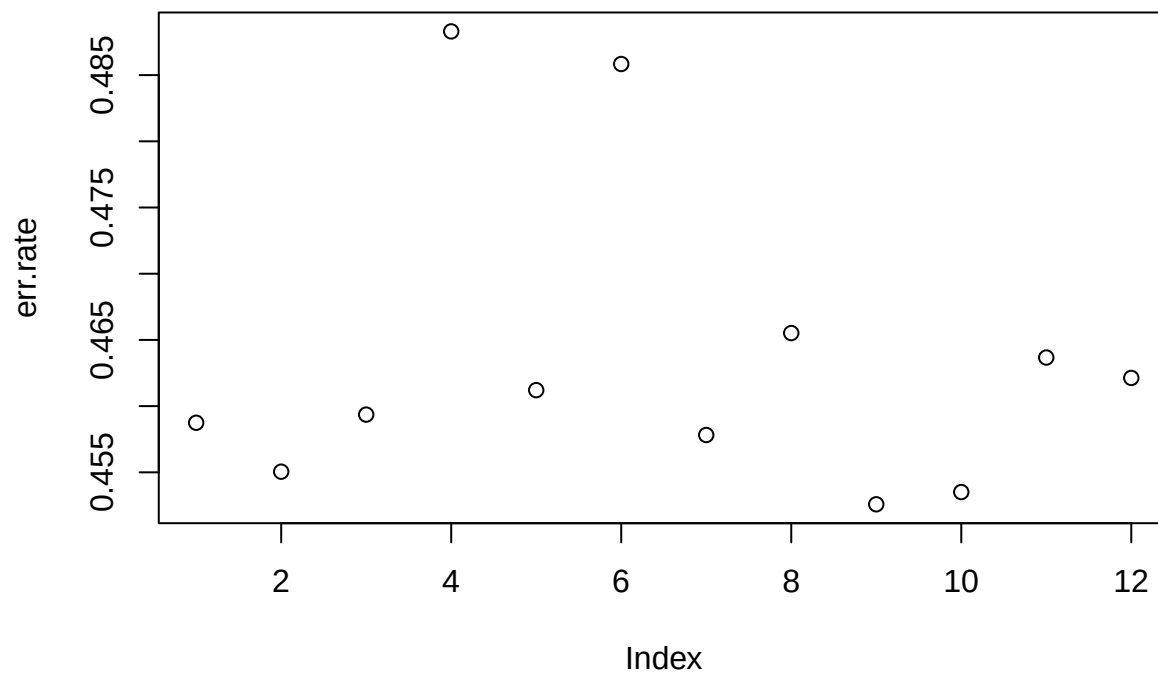


```
## stopped after 100 iterations
## # weights: 247
## initial value 11599.707194
## iter 10 value 4220.772420
## iter 20 value 4123.031903
## iter 30 value 4061.621204
## iter 40 value 4009.389936
## iter 50 value 3711.165221
## iter 60 value 3570.756933
## iter 70 value 3471.889656
## iter 80 value 3398.178730
## iter 90 value 3362.708132
## iter 100 value 3345.988627
## final value 3345.988627
## stopped after 100 iterations
```

```
err.rate
```

```
## [1] 0.4587438 0.4550493 0.4593596 0.4883005 0.4612069 0.4858374 0.4578202
## [8] 0.4655172 0.4525862 0.4535099 0.4636700 0.4621305
```

```
plot(err.rate)
```



9 hidden nodes produce the lowest prediction error rate

2. Cross Validation with Optimal number of hidden nodes

```
set.seed(1)
```

```
ann_wine_quality_optimal <- nnet(quality ~ .,
                                data = wine_train,
                                size = 9)
```

```
## # weights: 187
## initial value 7645.820596
## iter 10 value 4173.795018
## iter 20 value 4138.675802
## iter 30 value 4069.828752
## iter 40 value 3967.618915
## iter 50 value 3785.250404
## iter 60 value 3670.091805
## iter 70 value 3622.784786
## iter 80 value 3552.780655
## iter 90 value 3524.311480
## iter 100 value 3513.552929
## final value 3513.552929
## stopped after 100 iterations
```

```
yhat_ann <- predict(ann_wine_quality_optimal, newdata = wine_test, type = "class")
table(wine_test$quality, yhat_ann)
```

```
##      yhat_ann
##           3      5      6      7
## 3      0      5      8      1
## 4      0     69     31      1
## 5      1    639    422      3
## 6      1    330   1017     72
## 7      0     35    417     96
## 8      0      6     64     28
## 9      0      0      1      1
```

```
ann_classification_rate <- mean(wine_test$quality == yhat_ann)
mad_ann <- mean(abs(as.numeric(wine_test$quality) - as.numeric(yhat_ann)))
```

Summary: Which model performed at predicting the quality of a wine given its characteristics?

Using cross validation evaluating the performance of each model, the random forest model with 501 trees and 8 mtry produced the lowest error rate of 15.2%.

```
classification_rates <- data.frame(
  Model = c("K-Nearest Neighbors", "Classification Tree", "Random Forest", "Support Vector Machines", "ANN"),
  Classification_Rate = c(knn_cv_classification_rate,
                        tree_pruned_classification_rate,
                        forest_classification_rate,
                        svm_cv_classification_rate,
                        ann_classification_rate),
  Error_Rate = c(1 - knn_cv_classification_rate,
```

```

1- tree_pruned_classification_rate,
1- forest_classification_rate,
1- svm_cv_classification_rate,
1- ann_classification_rate),
Mean_Absolute_Deviation = c(mad_knn,
                             mad_tree,
                             mad_forest,
                             mad_svm,
                             mad_ann)
)

classification_rates %>%
  arrange(desc(Error_Rate))

```

```

##           Model Classification_Rate Error_Rate
## 1 Support Vector Machines      0.3682266 0.6317734
## 2      Classification Tree      0.5138547 0.4861453
## 3      K-Nearest Neighbors      0.5172414 0.4827586
## 4 Artificial Nerual Network      0.5394089 0.4605911
## 5      Random Forest          0.6490148 0.3509852
## Mean_Absolute_Deviation
## 1          0.8035714
## 2          0.5467980
## 3          0.6114532
## 4          1.9036330
## 5          0.3919335

```